```
Start coding or generate with AI.
```

```
import numpy as np

# --- 1. CONFIGURATION & PROBLEM SETUP ---
NUM_TASKS = 10
NUM_VMS = 3
POP_SIZE = 10        # Number of host nests (allocation plans)
PA = 0.25            # Probability of discovering a cuckoo egg
MAX_ITER = 50        # Total optimization loops

# VM capacities in MIPS (Million Instructions Per Second)
VM_MIPS = np.array([1000, 1500, 2000])
# Task lengths in MI (Million Instructions)
TASK_LENGTHS = np.array([4000, 2000, 6000, 3000, 1500, 5000, 7000, 2500, 4500, 3500])

# --- 2. FITNESS FUNCTION (MAKESPAN CALCULATOR) ---
def calculate_makespan(allocation):
    """
    Calculates the total time taken to finish all tasks under a specific plan.
    'allocation' is an array of size NUM_TASKS, where each element is a VM index (0, 1, or 2).
    """
    vm_times = np.zeros(NUM_VMS)

    # Accumulate execution time on each VM
    for task_idx, vm_idx in enumerate(allocation):
        execution_time = TASK_LENGTHS[task_idx] / VM_MIPS[vm_idx]
        vm_times[vm_idx] += execution_time

    # Makespan is determined by the bottleneck VM (the one that takes the longest)
    return np.max(vm_times)

# --- 3. LÉVY FLIGHT GENERATOR ---
def levy_flight(current_allocation):
    """
    Simulates a heavy-tailed random walk (Lévy Flight) to modify allocations.
    Randomly selects a few tasks and jumps them to different VMs.
    """
    new_allocation = current_allocation.copy()
    # Determine how many tasks to mutate (at least 1, up to half the tasks)
    num_mutations = np.random.randint(1, max(2, NUM_TASKS // 2))

    for _ in range(num_mutations):
        random_task = np.random.randint(0, NUM_TASKS)
        random_vm = np.random.randint(0, NUM_VMS)
        new_allocation[random_task] = random_vm

    return new_allocation

# --- 4. CUCKOO SEARCH ALGORITHM LOOP ---
# Step 1: Initialize random population of nests
# Each row represents a candidate allocation plan mapping 10 tasks to 3 VMs
nests = np.random.randint(0, NUM_VMS, size=(POP_SIZE, NUM_TASKS))
fitness = np.array([calculate_makespan(nest) for nest in nests])

# Track the global best solution found
best_idx = np.argmin(fitness)
best_nest = nests[best_idx].copy()
best_fitness = fitness[best_idx]

# Step 3: Run the iterative optimization
for iteration in range(MAX_ITER):

    # A. Generate a new solution via Lévy flight from a random nest
    random_cuckoo_idx = np.random.randint(0, POP_SIZE)
    new_cuckoo = levy_flight(nests[random_cuckoo_idx])
    new_fitness = calculate_makespan(new_cuckoo)

    # B. Compare with a random nest and replace if better
    target_nest_idx = np.random.randint(0, POP_SIZE)
    if new_fitness < fitness[target_nest_idx]:
        nests[target_nest_idx] = new_cuckoo
        fitness[target_nest_idx] = new_fitness

    # C. Abandon worst nests with probability Pa
    for i in range(POP_SIZE):
        if np.random.rand() < PA:
            # Replace with a brand new random allocation plan
            nests[i] = np.random.randint(0, NUM_VMS, size=NUM_TASKS)
            fitness[i] = calculate_makespan(nests[i])

    # D. Keep track of the absolute best plan across generations
```

```
        current_best_idx = np.argmin(fitness)
        if fitness[current_best_idx] < best_fitness:
            best_fitness = fitness[current_best_idx]
            best_nest = nests[current_best_idx].copy()

    # --- 5. OUTPUT RESULTS ---
    print("--- Optimization Complete ---")
    print(f"Optimal Task-to-VM Mapping: {best_nest}")
    print(f"Minimized Makespan Score  : {best_fitness:.2f} seconds")

    # Display breakdown
    for vm in range(NUM_VMS):
        assigned_tasks = [i for i, v in enumerate(best_nest) if v == vm]
        print(f"  VM {vm} handles Tasks: {assigned_tasks}")
```

```
--- Optimization Complete ---
Optimal Task-to-VM Mapping: [2 0 2 2 1 1 0 1 1 2]
Minimized Makespan Score  : 9.00 seconds
  VM 0 handles Tasks: [1, 6]
  VM 1 handles Tasks: [4, 5, 7, 8]
  VM 2 handles Tasks: [0, 2, 3, 9]
```